第 6 章：語言與有限狀態機

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

符號、有限字串與空字串

字母表 Σ 是有限非空符號集合；符號按不可再拆的 token 計，不把字形較長的標籤誤認成多個符號。字串 $w = a_1 \cdots a_n$ 是有限序列，長度 $|w| = n$。空字串 ε 長0，與一個空白符號不同。

$\Sigma^0 = \{\varepsilon\}$，$\Sigma^n$ 是長 n 的字串集合，$\Sigma^* = \bigcup_{n \geq 0} \Sigma^n$，$\Sigma^+ = \bigcup_{n \geq 1} \Sigma^n$。因此 $\Sigma^* \setminus \Sigma^+ = \{\varepsilon\}$，而 $\varnothing \neq \{\varepsilon\}$。

若 $|\Sigma| = s$，每個位置獨立選符號，故 $|\Sigma^n| = s^n$。先按長度、再按固定符號順序列舉，可列出所有字串，所以 Σ^* 可數無限；「每個字串有限」與「字串總數無限」可以同時成立。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

串接、前綴與反轉的精確物件

串接 uv 依序列先後排列，$|uv| = |u| + |v|$，$u\varepsilon = \varepsilon u = u$，且結合。一般不交換，例如 $01 \neq 10$。固定同一前綴可消去：$xu = xv \Rightarrow u = v$，因去掉前 $|x|$ 個位置後剩餘序列一致。

u 為 w 的前綴若 $w = uv$，後綴若 $w = vu$，子字串若 $w = xuy$。子字串需連續；子序列只需保留位置順序。例如01是001的前綴嗎？不是；它是後綴，也是子字串與子序列。

反轉滿足 $\varepsilon^R = \varepsilon$、$(aw)^R = w^R a$。對 $|u|$ 歸納：空前段成立；$u = av$ 時 $(uv')^R = (vv')^R a = (v')^R v^R a = (v')^R u^R$。因此 $(uv)^R = v^R u^R$，並由逐位置反兩次得 $(w^R)^R = w$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

語言串接不等於直積計數

語言是任意 $L \subseteq \Sigma^*$，可以空、有限或無限。定義 $AB = \{uv : u \in A, v \in B\}$。映射 $A \times B \to AB, (u, v) \mapsto uv$ 滿射，有限時只保證 $|AB| \leq |A||B|$，因分割點可能不唯一。

取 $A = \{a, aa\}, B = \{\varepsilon, a\}$，四個配對依序產生 a, aa, aa, aaa，所以 $AB = \{a, aa, aaa\}$ 只有3個元素。重複的是結果字串，不是配對。

$A\{\varepsilon\} = \{\varepsilon\}A = A$，而 $A\varnothing = \varnothing A = \varnothing$。語言串接一般不交換：$\{a\}\{b\} = \{ab\} \neq \{ba\}$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

語言代數的等號與單向包含

$(AB)C = A(BC)$，因兩側都是存在 $a \in A, b \in B, c \in C$ 的字串 abc。$A(B \cup C) = AB \cup AC$，因同一分解的後段在聯集中若且唯若在其中至少一個；右分配律同理。

交集只保證 $A(B \cap C) \subseteq AB \cap AC$：左側同一分解同時符合兩語言，右側卻容許不同分解。取 $A = \{\varepsilon, a\}, B = \{\varepsilon\}, C = \{a\}$，左空，右為 $\{a\}$，嚴格不等。

若 $A \subseteq B$，每個 A 因子也在 B，因此 $A^n \subseteq B^n$。這也可由 n 歸納：初始 $A^0 = B^0 = \{\varepsilon\}$，一步串接仍保包含。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Kleene star 的因子數不是字串長度

$L^0 = \{\varepsilon\}$、$L^{n+1} = LL^n$，$L^* = \bigcup_{n \geq 0} L^n$、$L^+ = \bigcup_{n \geq 1} L^n$。冪次 n 計串接幾個 L 中的字串，不一定是結果長度。

$\varnothing^* = \{\varepsilon\}$、$\varnothing^+ = \varnothing$，$\{\varepsilon\}^* = \{\varepsilon\}^+ = \{\varepsilon\}$。對一般語言，$L^+$ 可能含空字串；只有對字母表本身的 Σ^+ 才保證不含空字串。

L^* 包含 ε, L，且在串接下封閉：兩個有限因子列表接起來仍有限。反過來，任何含 ε, L 且串接封閉的集合，對因子數歸納都含每個 L^n，所以含 L^*。這證明 star 是滿足條件的最小集合。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Star 恆等式由分解與重組證明

$LL^* = L^*L = L^+$：非空因子列表可取第一個或最後一個因子；反向串接後仍至少一因子。因子本身為空字串也不影響此分類。

$L^*L^* = L^*$：合併兩個因子列表給左到右的包含；任一字串可在另一側串接空字串，給反向。類似展平有限個有限列表，得 $(L^*)^* = L^*$、$(L^*)^+ = L^*$、$(L^+)^* = L^*$。

$$(A \cup B)^* = (A^* \cup B^*)^* = (A^*B^*)^*.$$

第一個等號：把每個 A^* 或 B^* 因子展平，便是 $A \cup B$ 因子；反向每個單因子本來就在某一 star。第二個等號：每個 A^*B^* 可展成左右兩塊；反向把 A^* 因子寫成 $u\varepsilon$、B^* 因子寫成 εv。兩個包含都需證，不能從星號外觀直接約消。

遞迴語言的可靠性與完整性

由字串0開始，允許在左或右加1，並只保留有限次規則能生成的字串，得到恰含一個0的語言。可靠性：初始有一個0，每條規則不增減0。完整性：任一目標字串是 1^i01^j，從0先加 i 次左1、j 次右1即生成。

平衡括號語言 D 可用規則 $\varepsilon \in D$、$u, v \in D \Rightarrow (u)v \in D$ 定義。每個前綴的左括號數至少等於右括號，且全字串兩者相等。

可靠性由生成次數歸納：包住 u 時高度多1，完成後回0，再接 v，不會負。完整性對長度歸納：非空合法字串從左括號開始，找第一次高度回0的位置，唯一切成 $(u)v$；內部 u 與剩餘 v 都較短且合法，因此可生成。遞迴規則若沒有「最小生成集合」限制，可能容許未被規則真正生成的額外字串。

Mealy 機輸出的是整條串流

固定初態的 Mealy transducer（轉換器）寫作 $(Q, \Sigma, \Gamma, \delta, \lambda, q_0)$：狀態、輸入、輸出字母表有限，$\delta : Q \times \Sigma \to Q$、$\lambda : Q \times \Sigma \to \Gamma$ 都是全函數。每個輸入產生一個輸出符號，邊標為「輸入／輸出」。

定義擴充狀態與輸出：$\delta^*(q, \varepsilon) = q$，$\lambda^*(q, \varepsilon) = \varepsilon$，且

$$\delta^*(q, wa) = \delta(\delta^*(q, w), a), \quad \lambda^*(q, wa) = \lambda^*(q, w)\lambda(\delta^*(q, w), a).$$

輸出用讀該符號前的狀態計算，之後才轉到新狀態。把新狀態誤代入輸出函數，可能每一步都回錯。狀態有限，但輸入可任意長；記憶的是歷史的有限摘要，不是完整歷史。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

串列加法器只需記住進位

兩個二進位數由最低位開始輸入，每個符號是位元對 $(x, y) \in \{00, 01, 10, 11\}$。狀態 $c \in \{0, 1\}$ 是前一位的進位，初態0；令 $t = x + y + c$，輸出 $z = t \bmod 2$，新狀態 $c' = \lfloor t/2 \rfloor$。

舊進位	00 的新狀態／輸出	01	10	11
0	0／0	0／1	0／1	1／0
1	0／1	1／0	1／0	1／1

處理 k 個位元對後，令低位部分值為 X_k, Y_k, Z_k，不變量為 $X_k + Y_k = Z_k + 2^k c_k$。下一位等式 $x + y + c_k = z + 2c_{k+1}$ 乘 2^k 再相加，即維持不變量。初始各低位值為0，故證明涵蓋任意長輸入，不只表中八種單步。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

十三加六的每一位與最後進位

輸入 $13=(1101)_2$、$6=(0110)_2$，最低位先讀，最後補一組00排出進位。

位 k	輸入 (x_k,y_k)	舊 c	t	輸出 z_k	新 c
0	10	0	1	1	0
1	01	0	1	1	0
2	11	0	2	0	1
3	10	1	2	0	1
4，補位	00	1	1	1	0

輸出時間順序11001，反轉才是慣用最高位先寫的10011，即19。補位前不變量為 $13+6=3+16\cdot 1$，補位後進位0、輸出值19。若只讀原四位而忽略最終狀態，就只得到低四位3。

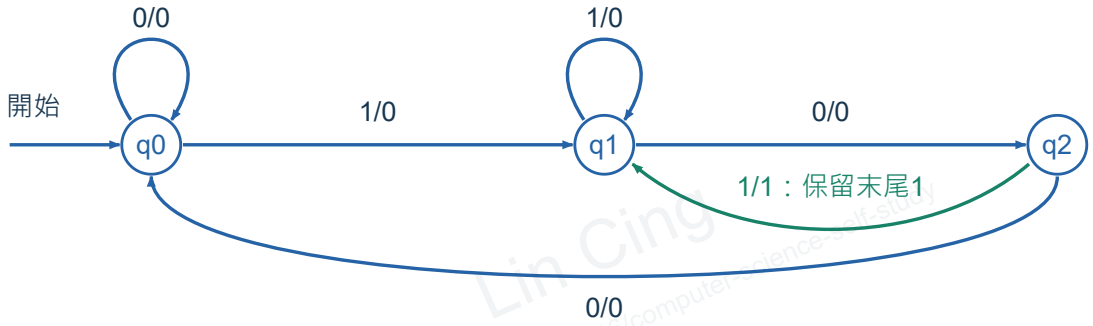
偵測101時必須保留重疊資訊

狀態 q_0,q_1,q_2 記目前字串後綴中，與101的真前綴相符的最長者：$\varepsilon,1,10$。每讀一位，若剛完成101就輸出1，否則0。

狀態	讀0：新態／輸出	讀1：新態／輸出
q_0	$q_0/0$	$q_1/0$
q_1	$q_2/0$	$q_1/0$
q_2	$q_0/0$	$q_1/1$

偵測101時必須保留重疊資訊 （續）

101 的重疊辨識：邊標為「輸入 / 輸出」



讀10101的狀態為 $q_0, q_1, q_2, q_1, q_2, q_1$，輸出00101。完成第一次101後回 q_1，因最後的1可開始下一次；若回 q_0，就漏掉位置3–5的重疊。表逐格保留最長相符後綴，配合空字串基例，對長度歸納即得每個輸出1若且唯若當下末三位是101。

加入位置限制需要額外相位

若只在位置為3的倍數時報告101，還須記已讀長度模3。使用 $(q, r) \in \{q_0, q_1, q_2\} \times \{0, 1, 2\}$，初態 $(q_0, 0)$。

讀 a 時先由原表得到 q', z，相位 $r' = (r + 1) \bmod 3$，輸出 $z' = 1$ 若且唯若 $z = 1 \wedge r' = 0$。新態 (q', r')。兩個分量不變量各保持，因此輸出精確符合兩條件。

10101原本輸出00101，限制後00100；101101則輸出001001。不能只在匹配成功時把計數歸零，否則相位會變成「距前次匹配的長度」，與全域位置不同。九個組合態是上界，不先假設每個都必要。

固定延遲的狀態數與輸出長度

延遲 $k \geq 1$ 個時脈，初始補0，可用狀態 $(b_1, \ldots, b_k) \in \{0,1\}^k$。讀 a 時輸出舊 b_1，新態為 $(b_2, \ldots, b_k, a)$。初態全0。

歸納可知佇列恰存最近 k 個輸入，不足的前面補0；所以第 t 個輸出在 $t \leq k$ 時為0，否則是第 $t-k$ 個輸入。輸入長 n 就輸出長 n，不是自動多輸出 k 位。

$k=2$ 讀10110：狀態00→01→10→01→11→10，輸出00101。若要排出最後兩位，需再讀兩個明確的補位。

2^k 個狀態也是下界：讀過任意不同的 k 位歷史後，接同一個 0^k；下一段輸出正是該歷史，因此這 2^k 種歷史不能共用同一狀態。

Mealy 與 Moore 的時序要先對齊

Moore 機的輸出為 $h : Q \to \Gamma$，只依狀態。此處採「每讀一符號、轉移後輸出新態的 h」；初態輸出另不列入字串。若採進入前輸出或包含初態輸出，會有一位的時序差，不能說兩個字串完全相等。

由 Moore 到 Mealy，設 $\lambda(q,a) = h(\delta(q,a))$，保留原轉移，每一步輸出相同。

由 Mealy 到 Moore，使用狀態 $(q,z) \in Q \times \Gamma$ 記新狀態及剛產生的輸出，令 $h(q,z) = z$，轉移到 $(\delta(q,a), \lambda(q,a))$。另設初態 s，第一次輸入 a 轉到 $(\delta(q_0,a), \lambda(q_0,a))$，$h(s)$ 不計入。

對已讀長度歸納，新狀態第一分量等於原 Mealy 狀態、第二分量等於最後輸出，所以非空輸入的整個輸出串一致，空輸入兩者均輸出空串。有限性保留，但狀態數可能增加。

可達、暫態、sink 與子機器

q 從 p 可達，若存在 $w \in \Sigma^*$ 使 $\delta^*(p, w) = q$，所以每態由空字串到達自己。若不存在非空 w 回到本態，稱暫態（transient）；不是「圖上沒有自迴圈」就夠，長度2的環也會回來。

Sink 滿足對每個符號 $\delta(q, a) = q$，因歸納也對全部字串不離開。子機器要求所選狀態對保留的輸入字母表封閉；只從圖上圈幾個節點，若仍有轉移指向外部，就不是該字母表下的子機器。

強連通要求任兩態互相可達。這些是轉移圖性質，不自動決定輸出；對接受器而言 sink 也不一定拒絕，它是否接受由接受集合決定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

最短 transfer sequence 的 BFS 證明

Transfer sequence 指從指定狀態到另一狀態的最短輸入字串。每符號成本1時，廣度優先搜尋（BFS）按字串長度逐層發現狀態；記每個新態的前驅與輸入符號，再反向回溯即可得到字串。

例轉移表：

態	0	1
S	A	B
A	A	T
B	T	B
T	T	T

由S開始，距離0為S，距離1為A、B，距離2首次找到T；01與10都是最短。重遇舊態不需再展開，因較長前綴不可能讓後續相同轉移變短。

若兩不同狀態可達，最短路不會重複節點，否則刪去環更短，所以長至多 $|Q| - 1$。查完整轉移表需 $O(|Q||\Sigma|)$ 時間、$O(|Q|)$ 搜尋空間；不同符號成本不等時這個 BFS 最短性不再適用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

接受器與串流輸出機器的差別

DFA（確定性有限自動機）為 $M=(Q,\Sigma,\delta,q_0,F)$，$q_0\in Q$、$F\subseteq Q$，$\delta$ 是全函數。接受條件是讀完 w 後 $\delta^*(q_0,w)\in F$，語言為 $L(M)$。

DFA 每個輸入只有一條完整路徑，讀完才判接受；中途經過接受態不代表整字串接受。空字串接受若且唯若 $q_0\in F$。雙圈標接受態、無來源箭頭標初態，兩者作用不同。

Mealy 則每步輸出，沒有內建接受集合。若用「最後輸出為1」當辨識條件，還需另定義空字串的結果；不能因機器尚無輸出就省掉空字串語意。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

擴充轉移的串接引理

$\delta^*(q,\varepsilon)=q$、$\delta^*(q,wa)=\delta(\delta^*(q,w),a)$。對所有字串 u,v，

$$\delta^*(q,uv)=\delta^*(\delta^*(q,u),v).$$

對 $|v|$ 歸納：空後段兩側相同；若 $v=xa$，左側先讀 ux 再讀 a，由歸納等於先讀 u、再讀 x、最後 a，即右側。

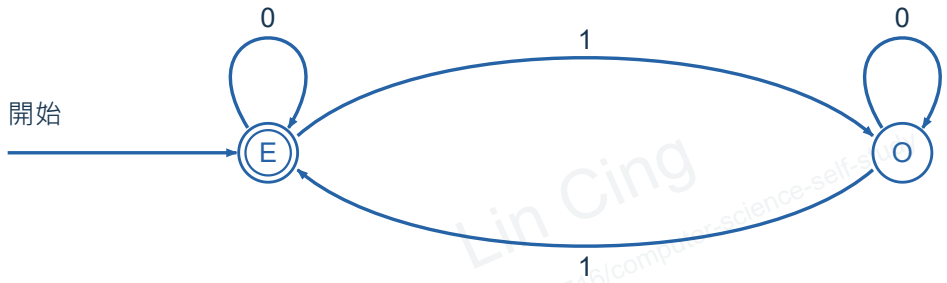
此引理使「相同狀態之後接相同後綴有相同結果」成為定理。設計機器需給狀態不變量，從空輸入開始逐符號證保持，最後證接受條件若且唯若目標性質；有限測試只能反駁錯設計，不能替代此全稱證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

偶數個1的兩態接受器

E, O 分別記已讀1的個數為偶、奇；初態E，$F = \{E\}$。讀0不變，讀1互換。

偶數個1：E 是初態及接受態；邊只標輸入



偶數個1　　奇數個1

空字串有0個1，初態不變量成立。追加0不改個數，追加1翻轉奇偶，正好符合四個轉移，歸納得到所有字串的狀態含義。因此接受若且唯若1個數為偶數。

1010的路徑為E→O→O→E→E，接受；1011最後O，拒絕。即使其中較早經過E，仍以最終狀態為準。

讀二進位數的模3摘要

允許前導0，並把空字串值定為0。若目前值為 v，追加位元 b 得 $2v + b$，所以狀態只需 $r = v \bmod 3$：

r	0	1
0	0	1
1	2	0
2	1	2

空值餘0，且同餘在乘2加位元後保持，所以歸納得最終r等於整字串數值模3。取初態0、接受集合 $\{0\}$，便辨識倍數。若要求非空表示法，還須額外記是否讀過至少一位；不能只改口頭語言而沿用接受空串的機器。

乘積機器同時保留兩個不變量

兩台同字母表全DFA，狀態用 $Q_1 \times Q_2$，初態 (q_{01}, q_{02})，轉移逐分量更新。歸納可得讀任意 w 後恰為 $(\delta_1^*(q_{01}, w), \delta_2^*(q_{02}, w))$。

接受集合選 $F_1 \times F_2$ 得交集；選 $(F_1 \times Q_2) \cup (Q_1 \times F_2)$ 得聯集；選「第一分量接受、第二拒絕」得差集，選恰一分量接受得對稱差。接受條件代入上述不變量，逐一得到語言等式。

狀態數至多 $|Q_1||Q_2|$，可從初態移除不可達組合。乘積不是先把整個字串交一台、再把輸出交另一台；兩台接受器讀的是同一個輸入。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

倍數且末位1的完整六態例

狀態 (r, ℓ) 記數值模3，以及末位是否1；空串取 $\ell = 0$。讀 b 更新成 $((2r + b) \bmod 3, b)$，初態(0,0)，唯一接受態(0,1)。

態	讀0	讀1
(0,0)	(0,0)	(1,1)
(0,1)	(0,0)	(1,1)
(1,0)	(2,0)	(0,1)
(1,1)	(2,0)	(0,1)
(2,0)	(1,0)	(2,1)
(2,1)	(1,0)	(2,1)

10101的狀態為(0,0)→(1,1)→(2,0)→(2,1)→(1,0)→(0,1)，值21所以接受。1010的值10且末位0，最終(1,0)，拒絕。空串也拒絕，因它沒有末位1。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

補集必須先補齊所有轉移

全DFA中每個字串恰到一個狀態，所以把 F 換成 $Q \setminus F$ 恰翻轉接受結果，補集相對於固定 Σ^*。

若允許部分轉移並約定缺邊即拒絕，直接翻轉雙圈會失敗。只有初態s為接受態、字母表 $\{a\}$、沒有邊時，原語言 $\{\varepsilon\}$；翻轉後沒有接受態，語言空，卻非正確補集 a^+。

先加入非接受sink d，所有缺邊導向d，且d每符號回自己；原語言不變。再翻轉使d接受、s拒絕，才得到 a^+。部分機器可以有明確拒絕語意，不應說缺邊時字串「既不接受也不拒絕」。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NFA 的接受是存在一條完整路徑

NFA（非確定性有限自動機）可採 $\Delta : Q \times (\Sigma \cup \{\varepsilon\}) \to \mathcal{P}(Q)$。一個符號可有零個、多個下一態；$\varepsilon$ 邊不消耗輸入。接受要求存在一條從初態出發、邊標去掉所有 ε 後恰為整個 w、終於接受態的有限路徑。

不是每條分支都要成功，也不是一抵達接受態就能忽略未讀輸入。沒有分支可走就拒絕；ε 環不要求實際無限走下去。

定義 $E(S)$ 為從 S 經零或多條 ε 邊可達的集合。用待處理佇列及visited集合，每次遇新態才加入，至多處理 $|Q|$ 態，故即使有環也終止，並由可達路徑長度歸納保證沒有漏態。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

子集建構與一般正確性

確定化的初態為 $S_0 = E(\{q_0\})$，每個DFA狀態是一個NFA狀態子集；讀 a 更新為

$$T(S,a) = E\left(\bigcup_{q\in S} \Delta(q,a)\right).$$

子集若交到原接受集合即為接受；空子集必保留為拒絕sink。對已讀字串長度歸納：空串可達集合正是 S_0；追加 a 的任何路徑先從現集合走一條a邊，再走任意 ε 邊，恰為更新式。反向每個更新點也由這種路徑可達。

因此讀完後子集含接受態若且唯若NFA有接受路徑，兩者語言相同。至多 $2^{|Q|}$ 個子集，但實作只展開可達者；NFA並未多辨識語言種類，而可能提供更精簡表示。

含 ε 邊的末尾10辨識器

初態s經 ε 到p；p對0、1自迴圈，另有1邊到q；q讀0到接受態r，其餘未列邊皆空。它猜測哪個1是最後兩位的起點，接受恰以10結尾的字串。

NFA 猜測最後兩位：存在一條完整讀完、終於 r 的路徑才接受



初態 s 只經 ε 邊起步；p 保留所有可能的前段。

未列出的邊 = 空集合；r 是唯一接受態。

含 ε 邊的末尾10辨識器 （續）

子集	讀0	讀1	接受？
S={s,p}	P={p}	Q={p,q}	否
P={p}	P	Q	否
Q={p,q}	R={p,r}	Q	否
R={p,r}	P	Q	是

1010走S→Q→R→Q→R，接受。中途R在追加1後不再接受，因舊r無出邊，但p保留另一個新匹配起點。0101最後Q，拒絕。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

聯集與串接的 NFA 建構

先把兩台機器狀態改成不交標籤。聯集加新初態，經 ε 分到兩初態，接受集合取兩者的聯集。每條接受路徑必選其中一台，反向任一台的接受路徑都可由新初態接上，所以語言恰為聯集。

串接保留第一台初態，從第一台每個接受態加 ε 到第二台初態，最終只接受第二台接受態。接受路徑可在跨機器的那條邊切成 uv，前段由第一台接受、後段由第二台接受；反向將兩條路徑接上即得。即使某段是空串，也由 ε 路徑正確處理。

改名使路徑不能從第二台意外回第一台。不要直接合併同名狀態，也不要把第一台接受集合一併留為整機接受，否則可能只讀完第一段就錯誤接受。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Star 與反轉也有明確路徑證明

Star 建構：加新初態s與新接受態t，s經 ε 到t及原初態；每個原接受態經 ε 到t及原初態，整機只以t接受。s→t代表零因子；其他接受路徑每次回原初態開始一個原語言因子，所以恰有限多因子串接。反向逐因子接原接受路徑即可。

反轉建構：所有邊倒向，保留原邊標；新初態經 ε 到每個原接受態，唯一接受態設原初態。一條原路徑倒走，非空符號次序反轉，故新語言是 $L^R = \{w^R : w \in L\}$，兩向都成立。原接受集合空時，新機沒有到原態的入口，語言仍空。

配合子集建構，已證正規語言對聯集、串接、star與反轉封閉；交集、差集、補集則由全DFA建構證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

正規表示式的語法與語意

正規表示式（regular expression）由 $\varnothing, \varepsilon, a\ (a \in \Sigma)$ 開始，有限次使用聯集 $R|S$、串接 RS、星號 R^*。其語意分別是空語言、只含空串、單符號語言，以及對應的語言運算。

優先序採star高於串接，高於聯集。$(0|1)^*10$ 表示任意二進位前段接10，正是前頁NFA語言；$0|1^*10$ 的括號不同，不是同一語言。

每個表示式都有等價NFA：三個基本項分別使用無接受路徑、單一 ε 路徑、單一a邊；聯集／串接／star使用前頁建構，對表示式構造樹作結構歸納，既證存在，也給出實際轉換程序。有限表示式使用有限次有限狀態建構，必終止。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

自動機轉回表示式的路徑遞迴

將NFA狀態編為1到n。$R_{ij}^{(k)}$ 表示從i到j、內部經過的狀態只許1到k的路徑標籤語言。基例 $R_{ij}^{(0)}$ 是所有直接邊標的聯集，$i=j$ 時另加 ε；沒有邊且不同點則 $\varnothing$。

$$R_{ij}^{(k)} = R_{ij}^{(k-1)} \mid R_{ik}^{(k-1)} \left(R_{kk}^{(k-1)}\right)^{*} R_{kj}^{(k-1)}.$$

證明：路徑若不以k為內部點，屬第一項；否則在每次訪問k切段，首段i到k、中間零或多段k到k、末段k到j，段內不再訪k，屬第二項。反向串接這些路徑只使用允許的內部點。端點本身為k時可用空首／末段，仍成立。

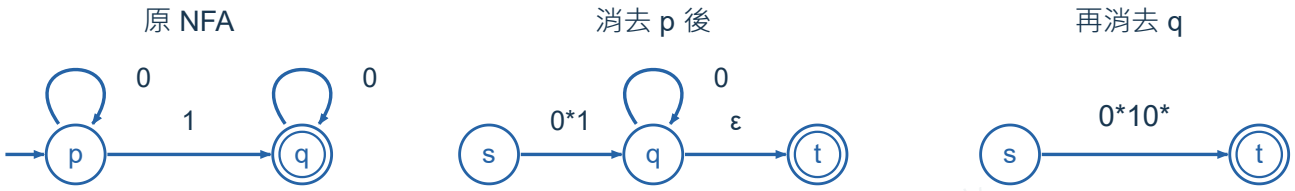
最後把初態到各接受態的 $R^{(n)}$ 聯集，就是整個接受語言的表示式。更新第k輪必用上一輪值，不能邊覆寫邊把尚未定義的第k輪結果混入。有限n輪終止，但展開的表示式可能很大，未聲稱轉換能保持原大小。

一次真正消去狀態的例子

兩態NFA：p為初態、q為接受態；p與q各有0自迴圈，p讀1到q，其餘無邊。添加新初態s經 ε 到p，q經 ε 到新接受態t。

一次真正消去狀態的例子 (續)

狀態消去：進入邊 × 自迴圈的 star × 離開邊



恰有一個1

s→q：ε 0* 1

s→t：0*1 0* ε

* 代表零或多次；因此字串1也必須接受。

消去p：s到q的新標籤為 $\varepsilon\,0^*1 = 0^*1$。再消去q：s到t標籤為 $(0^*1)0^*\varepsilon = 0^*10^*$。直接語意也核對：恰有一個1，前後任意個0。

若原本另有一條s到q邊，更新必取聯集，不能覆蓋它。自迴圈用star表示零或多次，漏掉零次就會錯拒字串1。

等價狀態比較所有未來後綴

同一DFA中 $p \equiv q$ 若對每個 $w \in \Sigma^*$，從p讀w與從q讀w的接受結果相同。它是等價關係；取 $w = \varepsilon$ 知兩態接受標記相同。

若 $p \equiv q$，對任意a、w，用串接引理將從 $\delta(p,a)$ 讀w改寫為從p讀aw，得 $\delta(p,a) \equiv \delta(q,a)$。所以可把每個等價類當一態，轉移 $[p] \xrightarrow{a} [\delta(p,a)]$ 與代表元無關；初態與接受類也有定義。

對輸入長度歸納，商機器最後到原最後態的類，接受語言不變。兩個態看起來鄰近、下一步標籤相同，都不足以替代全稱後綴條件；第7章再處理如何有效算出此分割。

兩台機器等價與最短反例

對兩台同字母表全DFA，建乘積，接受條件設兩分量恰有一個接受。語言是對稱差，所以兩台等價若且唯若乘積初態無法到達任何接受態。

用BFS列出所有可達乘積態，若找到接受態，回溯邊標得到最短區分字串。初態已接受時，反例就是空字串；否則最短路無重複態，反例長度至多 $|Q_1||Q_2| - 1$。若搜盡沒有接受態，所有字串的投影結果相同，證明等價。

例如只把偶數1機器的接受態E改為O，乘積初態已給不同接受結果，因此空字串就是最短反例。不需把長度上限內的所有字串暴力列舉才能下結論。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Pumping lemma 的量詞與一般證明

若語言由p態DFA接受，任意長 $|w| \geq p$ 的接受字串，都存在分解 $w = xyz$，使 $|xy| \leq p, |y| \geq 1$，且每個 $t \geq 0$ 都有 xy^tz 被接受。

證明：讀前p個符號會經過p+1個狀態位置，必有兩位置 $i < j \leq p$ 同態。令x為前i字、y為接下來j−i字、z為餘段；y是一個非空環，所以走零次或任意次，仍在同態接z，結果接受。

要反證正規，必對任意可能分解都能選t破壞；不能只挑自己喜歡的一種分解。引理是必要條件，滿足某些字串的可重複性不自動構造一台DFA，也不能當成充分條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

相同數量的0與1為何超過有限狀態

$L = \{0^n 1^n : n \geq 0\}$。假設由p態DFA接受，選 $w = 0^p 1^p$。任意符合 pumping 限制的y都位於前p個0中，且非空；取t=0會少掉至少一個0，1仍p個，違反相同數量，因此矛盾。

也可直接用可區分前綴：$\varepsilon, 0, \ldots, 0^p$ 中兩個 $0^i, 0^j$ $(i < j)$ 必到同態。接同一後綴 1^i 時，前者在L、後者不在，卻應同結果，矛盾。

這裡的不可能性針對固定有限態、任意長輸入。若先限制最多p個0，則可用有限多態記數，與上述結論不衝突。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

平衡括號可遞迴定義，卻非正規

對平衡括號語言D，前綴 $\varepsilon, (, ((, \ldots$ 具有任意多個尚未閉合括號。若 $(^i$ 與 $(^j$ $(i < j)$ 到同態，接同一 $)^i$ 後，前者平衡、後者仍有j−i個未閉合，矛盾。任意有限態數都會由鴿籠出現此碰撞，所以D非正規。

前面的遞迴建構仍正確，因遞迴規則並不等於有限狀態記憶。固定種類括號可用可成長的堆疊或計數資訊處理；將最大深度先限定為常數，才可把所有可能深度寫成有限態。

機器種類的界線由必須區分的歷史數目決定，不由圖畫得多複雜決定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

來源與模型範圍

Grimaldi, *Discrete and Combinatorial Mathematics*, 5th ed., §§6.1–6.3：字串／語言代數、遞迴定義、具輸出的有限狀態機、串列加法器、序列辨識、延遲、可達／暫態／子機器與transfer sequence。本章採原創輸入、狀態標籤與圖，並把接受器模型明確分開。

為銜接自動機理論，另給DFA／ε-NFA、子集建構、正規表示式雙向轉換、封閉性、商機器及非正規性的一般證明。輸出時序、空字串、缺邊與全部讀完的條件均按本章定義處理，不混用轉換器與接受器語意。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Ralph P. Grimaldi. Discrete and Combinatorial Mathematics: An Applied Introduction, 5th Edition, 2004.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study